

Experiment No: 10

Implement a program for solution of fractional Knapsack problem using greedy design technique.

Date:

Competency and Practical Skills: Algorithmic thinking, Programming Skills, Problem solving

Relevant CO: CO3

Objectives: (a) Understand greedy algorithm design method.
(b) Solve the optimization based problem.
(c) Find the time complexity of the algorithm.

Equipment/Instruments: Computer System, Any C language editor

Theory:

Knapsack problem is as stated below:

Given a set of items, each having different weight and value or profit associated with it. Find the set of items such that the total weight is less than or equal to a capacity of the knapsack and the total value earned is as large as possible.

Brute-force approach: The brute-force approach tries all the possible solutions with all the different fractions but it is a time-consuming approach.

Greedy approach: In Greedy approach, we calculate the ratio of profit/weight, and accordingly, we will select the item. The item with the highest ratio would be selected first.

Following are the steps to implement fractional knapsack using greedy design strategy.

1. Compute the value-to-weight ratio for each item in the knapsack.
2. Sort the items in decreasing order of value-to-weight ratio.
3. Initialize the total weight and total value to 0.
4. For each item in the sorted list:
 - a. If the entire item can be added to the knapsack without exceeding the weight capacity, add it and update the total weight and total value.
 - b. If the item cannot be added entirely, add a fraction of the item that fits into the knapsack and update the total weight and total value accordingly.
 - c. If the knapsack is full, stop the algorithm.
5. Return the total value and the set of items in the knapsack.

Implement the program based on the above logic for the solution of the fractional knapsack problem.

Observations:

Write observations based on whether this algorithm returns optimal answer or not on various inputs.

```
#include<stdio.h>
#include<conio.h>
void swap_Float(float *a , float *b){
    float temp;
    temp = *a;
    *a = *b;
    *b = temp;
```

```

}
void swap_Int(int *a , int *b){
    int temp = *a;
    *a = *b;
    *b = temp;
}
float KnapSack(int value[],int weight[],int n,int capacity){
    float ratio[n];
    float addedWeight = 0;
    float totalVal = 0;
    for(int i=0;i<n;i++){
        ratio[i] = (float)value[i] / weight[i];
    }
    for(int i=0;i<n-1;i++){
        for(int j=0;j<n-i-1;j++){
            if(ratio[j]<ratio[j+1]){
                swap_Float(&ratio[j],&ratio[j+1]);
                swap_Int(&value[j], &value[j+1]);
                swap_Int(&weight[j], &weight[j+1]);
            }
        }
    }
    for(int i=0;i<n;i++){
        if(addedWeight+weight[i] <= capacity){
            addedWeight+=weight[i];
            totalVal+=value[i];
        }else{
            int remain = capacity - addedWeight;
            totalVal+=value[i]*((float)remain/weight[i]);
            break;
        }
    }
    return totalVal;
}
int main(){
    int n,capacity;
    printf("Enter number of items : ");
    scanf("%d",&n);
    int value[n];
    int weight[n];
    for(int i=0;i<n;i++){
        printf("Enter %d item weight: ",i+1);
        scanf("%d",&weight[i]);
    }
    for(int i=0;i<n;i++){
        printf("Enter %d item value: ",i+1);
        scanf("%d",&value[i]);
    }
    printf("Enter capacity of KnapScak: ");
    scanf("%d",&capacity);
    float ans = KnapSack(value,weight,n,capacity);
    printf("Total Maximum Value : %f ",ans);
}

```

Result

```
● PS D:\ADA> gcc .\Practical10.c
● PS D:\ADA> ./a.exe
Enter number of items : 5
Enter 1 item weight: 80
Enter 2 item weight: 10
Enter 3 item weight: 40
Enter 4 item weight: 20
Enter 5 item weight: 90
Enter 1 item value: 40
Enter 2 item value: 10
Enter 3 item value: 50
Enter 4 item value: 30
Enter 5 item value: 60
Enter capacity of KnapScak: 120
Total Maximum Value : 123.333336
○ PS D:\ADA> 
```

Conclusion:

Quiz:

1. What is the time complexity of the above knapsack algorithm?

Answer: The time complexity is $O(n \log n)$ if sorting is used efficiently. In our simple implementation with nested loops, it is $O(n^2)$.

2. Does the above algorithm always return the optimal answer?

Answer: Yes, the Fractional Knapsack problem always gives the optimal answer with greedy approach, because fractions allow precise selection.

3. What is the time complexity solving knapsack problem using brute-force method?

Answer: The brute-force method checks all possible combinations, so **the time complexity is $O(2^n)$** , which is exponential and not practical for large inputs.

Suggested Reference:

1. "Introduction to Algorithms" by Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein
2. "Fundamentals of Algorithms" by E. Horowitz et al.

References used by the students:

Rubric wise marks obtained:

Rubrics	Understanding of problem (3)			Program Implementation (5)			Documentation & Timely Submission (2)			Total (10)
	Good (3)	Avg. (2-1)	Poor (1-0)	Good (5-4)	Avg. (3-2)	Poor (1-0)	Good (2)	Avg. (1)	Poor (0)	
Marks										